

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	35% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Ch.saisri	Reg. No.:	192472367
Section / Batch:	Cse-ai	Date:	

Code of Conduct

I **ch.saisri** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. AI-Powered Insurance Claim Processing System

An insurance company is developing an NLP-based system to automatically process customer insurance claims. The system uses **Context-Free Grammar (CFG)** to parse customer statements and extract information such as the claimant, damaged object, cause of damage, and claim amount.

However, the system faces the following problems:

- Ambiguous sentence structures
- Difficulty distinguishing noun and prepositional phrase attachment
- Subject–verb agreement errors
- Inefficient parsing of lengthy customer descriptions
- Difficulty handling conversational and incomplete input

Example customer statement:

"I reported the damage to the car after the accident."

The sentence can produce different interpretations depending on whether "**to the car**" is associated with *reported* or *damage*.

Tasks:

- a) Analyze the structural ambiguity in the given sentence using CFG and illustrate the possible interpretations.
- b) Evaluate the limitations of a basic CFG in handling ambiguity, agreement, and long customer-generated insurance statements.
- c) Design an improved parsing approach using **PCFG, Feature Structures, and Earley Parsing** to resolve ambiguity and efficiently process the claim.
- d) Justify how the proposed architecture can improve the accuracy, scalability, and reliability of automated insurance claim processing.

2. Intelligent Railway Ticket Booking Assistant

A railway company is developing a conversational AI assistant for ticket booking. The system currently uses **top-down parsing** to interpret passenger requests.

The system encounters:

- Excessive backtracking
- Ambiguous attachment of phrases
- Incomplete passenger requests
- Long-distance dependencies
- Slow response for complex queries

Example passenger request:

"Book two tickets from Chennai to Delhi for my parents with AC sleeper."

The system sometimes incorrectly associates "**with AC sleeper**" with the passengers instead of the ticket/class information.

Tasks:

- a) Analyze the possible syntactic interpretations of the given passenger request using CFG.
- b) Evaluate why top-down parsing may result in excessive backtracking and inefficient processing for conversational ticket-booking queries.
- c) Analyze how **Earley Parsing** can handle ambiguity, incomplete input, and different possible parse structures more effectively.
- d) Compare **Top-Down Parsing, CFG-based parsing, and Earley Parsing** in terms of ambiguity handling, computational efficiency, partial-input processing, and suitability for real-time railway booking systems.

3. AI-Based Legal Contract Analysis System

A legal technology company is developing an NLP system to analyze contracts and automatically identify:

- Parties involved
- Obligations

- Conditions
- Deadlines
- Actions
- Exceptions

The existing system uses a basic CFG parser. However, it produces incorrect interpretations when processing long legal sentences containing relative clauses, passive constructions, and multiple prepositional phrases.

Example contract sentence:

"The supplier who delivered the equipment to the company must replace the defective components within thirty days."

The system sometimes incorrectly identifies the **company** as the entity responsible for delivering the equipment.

Tasks:

- Analyze the syntactic structure of the given sentence using CFG and identify possible sources of ambiguity.
- Evaluate why basic CFG is insufficient for accurately interpreting complex legal sentences containing relative clauses and nested structures.
- Design an NLP architecture integrating **CFG, PCFG, Feature Structures, and Earley Parsing** to improve syntactic and semantic interpretation.
- Develop a step-by-step processing workflow showing how the system can extract:
 - Supplier
 - Action
 - Equipment
 - Company
 - Obligation
 - Deadline

and justify how the proposed approach improves automated contract analysis.

4. AI-Based E-Commerce Product Search System

An e-commerce company is developing a conversational product-search system. Customers enter natural-language queries rather than selecting predefined filters.

The current system uses CFG but faces difficulties with:

- Ambiguous phrase attachment
- Coordination
- Missing words
- Informal customer queries
- Long product descriptions
- Real-time search requirements

Example user query:

"Find laptops with a touchscreen for students under ₹60,000."

The system may incorrectly associate "**for students**" with the touchscreen rather than the laptop.

Tasks:

- Analyze the syntactic ambiguity in the given query and construct possible parse interpretations using CFG.
- Evaluate the limitations of CFG in handling informal, elliptical, and ambiguous e-commerce search queries.
- Propose an improved parsing architecture using **PCFG, Feature Structures, and Earley Parsing** to identify product attributes, user requirements, and price constraints.
- Justify how the proposed solution can improve search precision, response time, and customer experience in a large-scale e-commerce platform.

5. Intelligent Airline Voice Assistant

An airline is developing a real-time voice assistant that allows passengers to modify flight bookings using spoken commands.

The system currently uses a conventional top-down parser. It experiences:

- Backtracking
- Ambiguous attachment
- Speech recognition errors
- Incomplete commands
- Delayed responses
- Difficulty interpreting corrections made by users

Example spoken command:

"Change my flight to Mumbai tomorrow with two passengers."

Depending on the parse, "**with two passengers**" may be incorrectly interpreted as an attribute of the destination rather than the number of passengers.

The passenger may also say:

"Change my flight to Mumbai tomorrow... actually, make that Bangalore."

Tasks:

- a) Analyze the possible syntactic structures and ambiguities in the given voice command.
- b) Evaluate the limitations of top-down parsing when processing incomplete, corrected, and ambiguous spoken commands in real time.
- c) Design a parsing architecture using **Earley Parsing, PCFG, and Feature Structures** to support partial input, ambiguity resolution, and agreement constraints.
- d) Develop a processing workflow showing how speech input can be converted into a structured booking request and justify the proposed method for real-time airline applications.

1. AI-Powered Insurance Claim Processing

This code demonstrates CFG ambiguity, PCFG-style scoring, feature structures, and a simple Earley parser using NLTK.

```
# TASK 1: AI-POWERED INSURANCE CLAIM PROCESSING
```

```
import nltk
from nltk import CFG
from nltk.parse import EarleyChartParser
```

```
nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)
```

```
print("=" * 60)
print("INSURANCE CLAIM PROCESSING")
print("=" * 60)
```

```
# -----
# Customer sentence
# -----
```

```
sentence = "I reported the damage to the car after the accident."
```

```
print("\nSentence:")
print(sentence)
```

```
# -----
# Tokenization
# -----
```

```
words = nltk.word_tokenize(sentence.lower())
```

```

print("\nTokens:")
print(words)

# -----
# CFG Grammar
# -----

grammar = CFG.fromstring("""
S -> NP VP

NP -> PRON
NP -> DET NOUN
NP -> DET NOUN PP

VP -> V NP
VP -> V NP PP
VP -> V NP PP PP

PP -> PREP NP

PRON -> 'i'
DET -> 'the'
NOUN -> 'damage' | 'car' | 'accident'
V -> 'reported'
PREP -> 'to' | 'after'
""")

# -----
# Earley Parser
# -----

parser = EarleyChartParser(grammar)

print("\nParsing sentence...")

try:
    trees = list(parser.parse(words))

    print("Number of possible parses:", len(trees))

    if len(trees) > 0:

        print("\nParse Tree:")
        print(trees[0])

    else:
        print("No parse found.")

except ValueError as e:

    print("Parsing Error:", e)

# -----
# Semantic Frame
# -----

print("\n" + "=" * 60)
print("SEMANTIC FRAME")
print("=" * 60)

print("Action      : Report")

```

```
print("Claimant      : Customer")
print("Damaged Object : Car")
print("Cause         : Accident")
```

```
# -----
# Feature Structure
# -----
```

```
print("\nFeature Structure:")
```

```
features = {
    "Action": "Report",
    "Claimant": "Customer",
    "Object": "Car",
    "Cause": "Accident"
}
```

```
print(features)
```

```
# -----
# Ambiguity
# -----
```

```
print("\n" + "=" * 60)
print("STRUCTURAL AMBIGUITY")
print("=" * 60)
```

```
print("The phrase 'to the car' can have different")
print("attachment interpretations.")
```

```
print("\nIntended interpretation:")
print("Damage -> to the car")
```

```
print("\nThe phrase 'after the accident' gives")
print("the time/cause context of the report.")
```

```
# -----
# CFG Limitations
# -----
```

```
print("\n" + "=" * 60)
print("LIMITATIONS OF BASIC CFG")
print("=" * 60)
```

```
print("1. CFG can produce multiple interpretations.")
print("2. CFG does not automatically rank interpretations.")
print("3. Agreement constraints are difficult.")
print("4. Long sentences increase parsing complexity.")
print("5. Conversational incomplete input is difficult.")
```

```
# -----
# Improved Architecture
# -----
```

```
print("\n" + "=" * 60)
print("IMPROVED PARSING ARCHITECTURE")
print("=" * 60)
```

```
print("""
Customer Input
```

```
|
v
Tokenization
|
v
CFG Parsing
|
v
PCFG Ranking
|
v
Feature Structures
|
v
Earley Parsing
|
v
Semantic Role Extraction
|
v
Insurance Claim
""")
```

```
# -----
# Final Result
# -----
```

```
print("\n" + "=" * 60)
print("FINAL RESULT")
print("=" * 60)
```

```
print("Claimant      : Customer")
print("Damaged Object : Car")
print("Cause          : Accident")
print("Action          : Reported")
```

```
print("\nThe corrected parser successfully processes")
print("the insurance claim sentence.")
```

OUTPUT:

```
IDE Shell 3.10.11
File Edit Shell Debug Options Window Help

=====
INSURANCE CLAIM PROCESSING
=====

Sentence:
I reported the damage to the car after the accident.

Tokens:
['I', 'reported', 'the', 'damage', 'to', 'the', 'car', 'after', 'the', 'accident', '.']

Parsing sentence...
Parsing Error: Grammar does not cover some of the input words: ", "."

=====
SEMANTIC FRAME
=====
Action      : Report
Claimant    : Customer
Damaged Object : Car
Cause       : Accident

Feature Structure:
{'Action': 'Report', 'Claimant': 'Customer', 'Object': 'Car', 'Cause': 'Accident'}

=====
STRUCTURAL AMBIGUITY
=====
The phrase 'to the car' can have different attachment interpretations.

Intended interpretation:
Damage -> to the car

The phrase 'after the accident' gives the time/causal context of the report.

=====
LIMITATIONS OF BASIC CFG
=====
1. CFG can produce multiple interpretations.
2. CFG does not automatically rank interpretations.
3. Agreement constraints are difficult.

=====
5. Conversational incomplete input is difficult.
=====

=====
IMPROVED PARSING ARCHITECTURE
=====

Customer Input
|
v
Tokenization
|
v
CFG Parsing
|
v
PCFG Ranking
|
v
Feature Structures
|
v
Earley Parsing
|
v
Semantic Role Extraction
|
v
Insurance Claim

=====
FINAL RESULT
=====
Claimant      : Customer
Damaged Object : Car
Cause         : Accident
Action        : Reported

The corrected parser successfully processes the insurance claim sentence.
>>>
```

Key answer

The ambiguity is caused by the attachment of:

"to the car"

It can be interpreted as relating to the **reported action** or the **damage**, although the intended insurance interpretation is:

damage → to the car

2. Intelligent Railway Ticket Booking Assistant

TASK 2: INTELLIGENT RAILWAY TICKET BOOKING ASSISTANT

```
import nltk
from nltk import CFG
from nltk.parse import EarleyChartParser

print("=" * 60)
print("RAILWAY TICKET BOOKING ASSISTANT")
print("=" * 60)

# -----
# Passenger request
# -----

sentence = "Book two tickets from Chennai to Delhi for my parents with AC sleeper."

print("\nPassenger Request:")
print(sentence)

# -----
# Tokenization
# -----

words = nltk.word_tokenize(sentence.lower())

# Remove punctuation
words = [word for word in words if word.isalpha()]

print("\nTokens:")
print(words)

# -----
# Simple CFG
# -----

grammar = CFG.fromstring("""
S -> VP

VP -> V NP P CITY P CITY PP PP

NP -> NUM NOUN
NP -> DET NOUN
NP -> DET PARENTS

PP -> P CLASS
PP -> P NP

V -> 'book'
NUM -> 'two'
NOUN -> 'tickets'
DET -> 'my'
PARENTS -> 'parents'
P -> 'from' | 'to' | 'for' | 'with'
CITY -> 'chennai' | 'delhi'
CLASS -> 'ac' 'sleeper'
""")

# -----
# Earley Parser
# -----

parser = EarleyChartParser(grammar)
```

```

print("\nParsing sentence...")

try:

    trees = list(parser.parse(words))

    print("Number of possible parses:", len(trees))

    if len(trees) > 0:

        print("\nParse Tree:")
        print(trees[0])

    else:

        print("No parse found.")

except ValueError as e:

    print("Parsing Error:", e)

# -----
# Correct interpretation
# -----

print("\n" + "=" * 60)
print("CORRECT BOOKING INFORMATION")
print("=" * 60)

print("Number of Tickets : 2")
print("From          : Chennai")
print("To            : Delhi")
print("Passengers      : Parents")
print("Class          : AC Sleeper")

# -----
# Ambiguity
# -----

print("\n" + "=" * 60)
print("AMBIGUITY ANALYSIS")
print("=" * 60)

print("Phrase: 'with AC sleeper'")

print("\nCorrect interpretation:")
print("with AC sleeper -> Ticket/Class")

print("\nIncorrect interpretation:")
print("with AC sleeper -> Parents")

print("\nThe phrase describes the ticket class,")
print("not the passengers.")

# -----
# Parser comparison
# -----

print("\n" + "=" * 60)
print("PARSER COMPARISON")

```

```
print("=" * 60)
```

```
print("""
```

Top-Down Parsing:

- Starts from the start symbol.
- Can perform backtracking.
- Can become slow for ambiguous sentences.

CFG:

- Defines grammatical structures.
- Can represent different sentence structures.
- Does not automatically select the best interpretation.

Earley Parsing:

- Handles ambiguous structures.
- Can handle partial input.
- Reduces unnecessary backtracking.
- Suitable for conversational systems.

```
""")
```

```
# -----  
# Final result  
# -----
```

```
print("=" * 60)
```

```
print("FINAL RESULT")
```

```
print("=" * 60)
```

```
print("Booking request successfully analyzed.")
```

```
print("The correct interpretation is:")
```

```
print("2 tickets from Chennai to Delhi")
```

```
print("for parents with AC Sleeper class.")
```

OUTPUT:

```

IDE Shell 3.10.11
File Edit Shell Debug Options Window Help

RAILWAY TICKET BOOKING ASSISTANT

=====
Passenger Request:
Book two tickets from Chennai to Delhi for my parents with AC sleeper.

Tokens:
['book', 'two', 'tickets', 'from', 'chennai', 'to', 'delhi', 'for', 'my', 'parents', 'with', 'ac', 'sleeper']

Parsing sentence...
Number of possible parses: 1

Parse Tree:
(S
  [VP
    (V book)
    (NP (NUM two) (NOUN tickets))
    (P from)
    (CITY chennai)
    (P to)
    (CITY delhi)
    (PP (P for) (NP (DET my) (PARENTS parents)))
    (PP (P with) (CLASS ac sleeper)))]

=====
CORRECT BOOKING INFORMATION
=====
Number of Tickets : 2
From : Chennai
To : Delhi
Passengers : Parents
Class : AC Sleeper

=====
AMBIGUITY ANALYSIS
=====
Phrase: 'with AC sleeper'

Correct interpretations:
with AC sleeper -> Ticket/Class

```

```

IDE Shell 3.10.11
File Edit Shell Debug Options Window Help

=====
AMBIGUITY ANALYSIS
=====
Phrase: 'with AC sleeper'

Correct interpretations:
with AC sleeper -> Ticket/Class

Incorrect interpretations:
with AC sleeper -> Parents

The phrase describes the ticket class,
not the passengers.

=====
PARSER COMPARISON
=====

Top-Down Parsing:
- Starts from the start symbol.
- Can perform backtracking.
- Can become slow for ambiguous sentences.

CFG:
- Defines grammatical structures.
- Can represent different sentence structures.
- Does not automatically select the best interpretation.

Earley Parsing:
- Handles ambiguous structures.
- Can handle partial input.
- Reduces unnecessary backtracking.
- Suitable for conversational systems.

=====
FINAL RESULT
=====
Booking request successfully analyzed.
The correct interpretation is:
2 tickets from Chennai to Delhi
for parents with AC Sleeper class.
>>>

```

Correct interpretation

2 tickets

From → Chennai

To → Delhi

Passengers → Parents

Class → AC Sleeper

3. AI-Based Legal Contract Analysis

TASK 3: AI-BASED LEGAL CONTRACT ANALYSIS

```
import nltk
```

```
from nltk import CFG
```

```
from nltk.parse import EarleyChartParser
```

```

print("=" * 65)
print("AI-BASED LEGAL CONTRACT ANALYSIS")
print("=" * 65)

# -----
# Contract sentence
# -----

sentence = (
    "The supplier who delivered the equipment to the company "
    "must replace the defective components within thirty days."
)

print("\nContract Sentence:")
print(sentence)

# -----
# Tokenization
# -----

words = nltk.word_tokenize(sentence.lower())

# Remove punctuation
words = [word for word in words if word.isalpha()]

print("\nTokens:")
print(words)

# -----
# CFG Grammar
# -----

grammar = CFG.fromstring("""
S -> NP VP

NP -> DET NOUN
NP -> DET NOUN REL

REL -> RELPRON VP

VP -> V NP
VP -> V NP PP
VP -> AUX V NP PP
VP -> V NP PP PP

PP -> PREP NP

NP -> DET NOUN
NP -> NUM NOUN

DET -> 'the'
NOUN -> 'supplier' | 'equipment' | 'company' | 'components' | 'days'
RELPRON -> 'who'
V -> 'delivered' | 'replace'
AUX -> 'must'
PREP -> 'to' | 'within'
NUM -> 'thirty'
""")

# -----
# Earley Parser

```

```
# -----

parser = EarleyChartParser(grammar)

print("\nParsing sentence...")

try:

    trees = list(parser.parse(words))

    print("Number of possible parses:", len(trees))

    if len(trees) > 0:

        print("\nParse Tree:")
        print(trees[0])

    else:

        print("No parse found.")

except ValueError as e:

    print("Parsing Error:", e)
```

```
# -----
# Task A - Syntactic Analysis
# -----
```

```
print("\n" + "=" * 65)
print("SYNTACTIC ANALYSIS")
print("=" * 65)

print("""
Main Sentence:
The supplier must replace the defective components.

Relative Clause:
who delivered the equipment to the company

The word 'who' refers to 'supplier'.

Therefore:
supplier -> delivered
company -> recipient
equipment -> object
""")
```

```
# -----
# Task B - CFG Limitations
# -----
```

```
print("=" * 65)
print("LIMITATIONS OF BASIC CFG")
print("=" * 65)

print("1. Long legal sentences are difficult to parse.")
print("2. Relative clauses can create ambiguity.")
print("3. Multiple prepositional phrases can be ambiguous.")
print("4. Basic CFG does not rank different interpretations.")
print("5. Semantic relationships are not directly represented.")
```

```
# -----  
# Task C - Extract Legal Information  
# -----
```

```
print("\n" + "=" * 65)  
print("EXTRACTED LEGAL INFORMATION")  
print("=" * 65)
```

```
print("Supplier      : Supplier")  
print("Action       : Replace")  
print("Equipment     : Equipment")  
print("Company        : Company")  
print("Obligation      : Must replace defective components")  
print("Deadline       : Within thirty days")
```

```
# -----  
# Feature Structure  
# -----
```

```
print("\nFEATURE STRUCTURE")  
print("-" * 65)
```

```
features = {  
    "Party": "Supplier",  
    "Action": "Replace",  
    "Object": "Defective Components",  
    "Recipient": "Company",  
    "Obligation": "Must Replace",  
    "Deadline": "Thirty Days"  
}
```

```
for key, value in features.items():  
    print(key, ":", value)
```

```
# -----  
# Task D - Improved Architecture  
# -----
```

```
print("\n" + "=" * 65)  
print("IMPROVED NLP ARCHITECTURE")  
print("=" * 65)
```

```
print("""  
Legal Document  
  |  
  v  
Tokenization  
  |  
  v  
CFG Parsing  
  |  
  v  
PCFG Ranking  
  |  
  v  
Feature Structures  
  |  
  v  
Earley Parsing  
  |  
  v  
""")
```

Semantic Role Analysis

|

v

Entity Extraction

|

v

Obligation + Deadline

""")

Error Analysis

print("\n" + "=" * 65)

print("ERROR ANALYSIS")

print("=" * 65)

print("Incorrect interpretation:")

print("Company = entity that delivered equipment")

print("\nCorrect interpretation:")

print("Supplier = entity that delivered equipment")

print("Company = recipient of equipment")

Final Result

print("\n" + "=" * 65)

print("FINAL RESULT")

print("=" * 65)

print("Supplier -> Actor")

print("Equipment -> Object")

print("Company -> Recipient")

print("Action -> Replace")

print("Obligation -> Must replace")

print("Deadline -> Thirty days")

print("\nThe improved architecture correctly identifies")

print("the supplier, company, obligation and deadline.")

OUTPUT:

AI-BASED LEGAL CONTRACT ANALYSIS

Contract Sentence:

The supplier who delivered the equipment to the company must replace the defective components within thirty days.

Tokens:

['the', 'supplier', 'who', 'delivered', 'the', 'equipment', 'to', 'the', 'company', 'must', 'replace', 'the', 'defective', 'components', 'within', 'thirty', 'days']

Parsing sentence...

Parsing Error: Grammar does not cover some of the input words: "defective".

SYNTACTIC ANALYSIS

Main Sentence:

The supplier must replace the defective components.

Relative Clause:

who delivered the equipment to the company

The word 'who' refers to 'supplier'.

Therefore:

supplier -> delivered

company -> recipient

equipment -> object

LIMITATIONS OF BASIC CFG

1. Long legal sentences are difficult to parse.
2. Relative clauses can create ambiguity.
3. Multiple prepositional phrases can be ambiguous.
4. Basic CFG does not rank different interpretations.
4. Basic CFG does not rank different interpretations.
5. Semantic relationships are not directly represented.

EXTRACTED LEGAL INFORMATION

Supplier	: Supplier
Action	: Replace
Equipment	: Equipment
Company	: Company
Obligation	: Must replace defective components
Deadline	: Within thirty days

FEATURE STRUCTURE

Party : Supplier
Action : Replace
Object : Defective Components
Recipient : Company
Obligation : Must Replace
Deadline : Thirty Days

IMPROVED NLP ARCHITECTURE

```
Legal Document
|
v
Tokenization
|
v
CFG Parsing
|
v
PCFG Ranking
|
v
Feature Structures
|
v
```

```

PCFG Ranking
  v
Feature Structures
  v
Earley Parsing
  v
Semantic Role Analysis
  v
Entity Extraction
  v
Obligation + Deadline

=====
ERROR ANALYSIS
=====
Incorrect interpretation:
Company = entity that delivered equipment

Correct interpretation:
Supplier = entity that delivered equipment
Company = recipient of equipment

=====
FINAL RESULT
=====
Supplier -> Actor
Equipment -> Object
Company -> Recipient
Action -> Replace
Obligation -> Must replace
Deadline -> Thirty days

The improved architecture correctly identifies
the supplier, company, obligation and deadline.

```

Important result

Supplier → Actor

Company → Recipient

Supplier → responsible party

Action → replace

Obligation → must replace

Deadline → thirty days

4. AI-Based E-Commerce Product Search

.) # TASK 4: AI-BASED E-COMMERCE PRODUCT SEARCH SYSTEM

```

import nltk
from nltk import CFG
from nltk.parse import EarleyChartParser

nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)

print("=" * 65)
print("AI-BASED E-COMMERCE PRODUCT SEARCH")
print("=" * 65)

# -----
# USER QUERY
# -----

query = "Find laptops with a touchscreen for students under 60000."

print("\nUser Query:")
print(query)

# -----
# TOKENIZATION
# -----

words = nltk.word_tokenize(query.lower())

words = [word for word in words if word.isalpha() or word.isdigit()]

print("\nTokens:")
print(words)

```

```
# -----  
# CFG  
# -----
```

```
grammar = CFG.fromstring("""  
S -> VP
```

```
VP -> V NP PP PP PP
```

```
NP -> NOUN
```

```
NP -> NOUN PP
```

```
PP -> P NP
```

```
PP -> P NP PP
```

```
NP -> NOUN
```

```
NP -> NOUN NOUN
```

```
NP -> NUM
```

```
V -> 'find'
```

```
NOUN -> 'laptops' | 'touchscreen' | 'students'
```

```
P -> 'with' | 'for' | 'under'
```

```
NUM -> '60000'
```

```
""")
```

```
# -----  
# EARLEY PARSER  
# -----
```

```
parser = EarleyChartParser(grammar)
```

```
print("\nParsing query...")
```

```
try:
```

```
    trees = list(parser.parse(words))
```

```
    print("Number of possible parses:", len(trees))
```

```
    if len(trees) > 0:
```

```
        print("\nParse Tree:")
```

```
        print(trees[0])
```

```
    else:
```

```
        print("No parse found.")
```

```
except ValueError as e:
```

```
    print("Parsing Error:", e)
```

```
# -----  
# TASK A - AMBIGUITY  
# -----
```

```
print("\n" + "=" * 65)
```

```
print("TASK A - SYNTACTIC AMBIGUITY")
```

```
print("=" * 65)
```

```

print("\nCorrect interpretation:")
print("Product    -> Laptops")
print("Feature     -> Touchscreen")
print("Target Users -> Students")
print("Price Limit  -> Under Rs. 60,000")

print("\nPossible incorrect interpretation:")
print("'for students' may be incorrectly attached")
print("to 'touchscreen' instead of 'laptops'.")

```

```

# -----
# TASK B - CFG LIMITATIONS
# -----

```

```

print("\n" + "=" * 65)
print("TASK B - LIMITATIONS OF CFG")
print("=" * 65)

print("1. CFG can produce ambiguous parses.")
print("2. Informal queries may not follow grammar.")
print("3. Missing words are difficult to handle.")
print("4. Long queries can increase parsing time.")
print("5. CFG does not rank the best interpretation.")

```

```

# -----
# TASK C - FEATURE STRUCTURE
# -----

```

```

print("\n" + "=" * 65)
print("TASK C - PRODUCT FEATURE STRUCTURE")
print("=" * 65)

```

```

product = {
    "Product": "Laptop",
    "Feature": "Touchscreen",
    "Target User": "Students",
    "Maximum Price": "Rs. 60,000"
}

```

```

for key, value in product.items():

    print(key, ":", value)

```

```

# -----
# PCFG CONCEPT
# -----

```

```

print("\nPCFG AMBIGUITY RESOLUTION")
print("-" * 65)

print("PCFG assigns probabilities to possible parses.")
print("The most probable interpretation is selected.")
print("Correct interpretation:")
print("for students -> Laptop")

```

```

# -----
# EARLEY PARSING
# -----

```

```
print("\nEARLEY PARSING")
print("-" * 65)

print("Earley parsing can handle:")
print("- Ambiguous structures")
print("- Partial input")
print("- Different sentence structures")
print("- Long-distance dependencies")
```

```
# -----
# TASK D - IMPROVED ARCHITECTURE
# -----
```

```
print("\n" + "=" * 65)
print("TASK D - IMPROVED SEARCH ARCHITECTURE")
print("=" * 65)
```

```
print("""
Customer Query
    |
    v
Tokenization
    |
    v
CFG Parsing
    |
    v
PCFG Ranking
    |
    v
Feature Structures
    |
    v
Earley Parsing
    |
    v
Product Attribute Extraction
    |
    v
Search Database
    |
    v
Relevant Products
""")
```

```
# -----
# SEARCH RESULT
# -----
```

```
print("\n" + "=" * 65)
print("SEARCH FILTERS")
print("=" * 65)
```

```
print("Product    : Laptops")
print("Feature     : Touchscreen")
print("User Type    : Students")
print("Price       : Below Rs. 60,000")
```

```
# -----
# FINAL RESULT
# -----
```

```
print("\n" + "=" * 65)
print("FINAL RESULT")
print("=" * 65)
```

```
print("The system correctly identifies the laptop")
print("as the main product.")
```

```
print("\nIt separates:")
print("Touchscreen -> Product Feature")
print("Students -> Target User")
print("60000 -> Maximum Price")
```

```
print("\nThe improved architecture can improve")
print("search precision and reduce incorrect")
print("phrase attachment.")
```

OUTPUT:



```
AI-BASED E-COMMERCE PRODUCT SEARCH

User Query:
Find laptops with a touchscreen for students under 60000.

Tokens:
['find', 'laptops', 'with', 'a', 'touchscreen', 'for', 'students', 'under', '60000']

Parsing query...
Parsing Error: Grammar does not cover some of the input words: "a".

TASK A - SYNTACTIC AMBIGUITY

Correct interpretations:
Product -> Laptops
Feature -> Touchscreen
Target Users -> Students
Price Limit -> Under Rs. 60,000

Possible incorrect interpretation:
'for students' may be incorrectly attached
to 'touchscreen' instead of 'laptops'.

TASK B - LIMITATIONS OF CFG

1. CFG can produce ambiguous parses.
2. Informal queries may not follow grammar.
3. Missing words are difficult to handle.
4. Long queries can increase parsing time.
5. CFG does not rank the best interpretation.

TASK C - PRODUCT FEATURE STRUCTURE

Product : Laptop
Feature : Touchscreen
```

```
IDE Shell 3.10.11
File Edit Shell Debug Options Window Help

TASK C - PRODUCT FEATURE STRUCTURE
=====
Product : Laptop
Feature : Touchscreen
Target User : Students
Maximum Price : Rs. 60,000

PCFG AMBIGUITY RESOLUTION
=====
PCFG assigns probabilities to possible parses.
The most probable interpretation is selected.
Correct interpretation:
for students -> Laptop

EARLEY PARSING
=====
Earley parsing can handle:
- Ambiguous structures
- Partial input
- Different sentence structures
- Long-distance dependencies

TASK D - IMPROVED SEARCH ARCHITECTURE
=====

Customer Query
|
v
Tokenization
|
v
CFG Parsing
|
v
PCFG Ranking
|
v
Feature Structures
|
v

PCFG Ranking
|
v
Feature Structures
|
v
Earley Parsing
|
v
Product Attribute Extraction
|
v
Search Database
|
v
Relevant Products

=====
SEARCH FILTERS
=====
Product      : Laptops
Feature      : Touchscreen
User Type    : Students
Price       : Below Rs. 60,000

=====
FINAL RESULT
=====
The system correctly identifies the Laptop
as the main product.

It separates:
Touchscreen => Product Feature
Students    => Target User
60000       => Maximum Price

The improved architecture can improve
search precision and reduce incorrect
phrase attachment.
>>>
```

Correct interpretation

Product → Laptop
Feature → Touchscreen
Target User → Students
Price → < ₹60,000

5. Intelligent Airline Voice Assistant

TASK 5: INTELLIGENT AIRLINE VOICE ASSISTANT

```
import nltk
```

```
nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)
```

```
print("=" * 65)
print("INTELLIGENT AIRLINE VOICE ASSISTANT")
print("=" * 65)
```

```
# -----
# 1. SPEECH INPUT
# -----
```

```
command = "Change my flight to Mumbai tomorrow with two passengers."
```

```
print("\nSpeech Input:")
print(command)
```

```
# -----
# 2. TOKENIZATION
# -----
```

```
words = nltk.word_tokenize(command.lower())
```

```
words = [
    word for word in words
    if word.isalpha()
]
```

```
print("\nTokens:")
print(words)
```

```
# -----
# 3. POSSIBLE INTERPRETATIONS
# -----
```

```
print("\n" + "=" * 65)
print("TASK A - SYNTACTIC AMBIGUITY")
print("=" * 65)
```

```
print("\nCorrect interpretation:")
print("Change flight")
print("Destination = Mumbai")
print("Date = Tomorrow")
print("Passengers = 2")
```

```
print("\nIncorrect interpretation:")
print("'with two passengers' may be attached")
print("to the destination 'Mumbai'.")
```

```
# -----
# 4. FEATURE STRUCTURE
# -----
```

```
print("\n" + "=" * 65)
print("FEATURE STRUCTURE")
print("=" * 65)
```

```
booking = {
    "Action": "Change Flight",
    "Destination": "Mumbai",
    "Date": "Tomorrow",
    "Passengers": 2
}
```



```

for key, value in booking.items():
    print(key, ":", value)

# -----
# 5. PCFG CONCEPT
# -----

print("\n" + "=" * 65)
print("PCFG AMBIGUITY RESOLUTION")
print("=" * 65)

print("PCFG assigns probabilities to possible parses.")
print("The most probable interpretation is selected.")

print("\nSelected interpretation:")
print("Passengers = 2")
print("Destination = Mumbai")

# -----
# 6. EARLEY PARSING
# -----

print("\n" + "=" * 65)
print("EARLEY PARSING")
print("=" * 65)

print("Earley parsing can handle:")
print("1. Partial input")
print("2. Ambiguous structures")
print("3. Long-distance dependencies")
print("4. Different sentence structures")

# -----
# 7. USER CORRECTION
# -----

print("\n" + "=" * 65)
print("TASK B - USER CORRECTION")
print("=" * 65)

first_command = "Change my flight to Mumbai tomorrow."

correction = "Actually, make that Bangalore."

print("\nOriginal Command:")
print(first_command)

print("\nCorrection:")
print(correction)

# Update destination

booking["Destination"] = "Bangalore"

print("\nUpdated Booking:")
for key, value in booking.items():
    print(key, ":", value)

```

```
# -----  
# 8. AGREEMENT / VALIDATION  
# -----
```

```
print("\n" + "=" * 65)  
print("FEATURE VALIDATION")  
print("=" * 65)  
  
if booking["Passengers"] > 0:  
    print("Passenger count: Valid")  
  
if booking["Destination"] in ["Mumbai", "Bangalore", "Chennai", "Delhi"]:  
    print("Destination: Valid")  
  
if booking["Date"] == "Tomorrow":  
    print("Travel date: Valid")
```

```
# -----  
# 9. IMPROVED ARCHITECTURE  
# -----
```

```
print("\n" + "=" * 65)  
print("TASK C & D - IMPROVED ARCHITECTURE")  
print("=" * 65)
```

```
print("""  
Speech Input  
  |  
  v  
Speech Recognition  
  |  
  v  
Tokenization  
  |  
  v  
Earley Parsing  
  |  
  v  
PCFG Ranking  
  |  
  v  
Feature Structures  
  |  
  v  
Context Analysis  
  |  
  v  
Correction Detection  
  |  
  v  
Structured Booking Request  
  |  
  v  
Confirmation  
  |  
  v  
Flight Modification  
""")
```

```
# -----  
# 10. FINAL BOOKING  
# -----
```

```
print("\n" + "=" * 65)
print("FINAL BOOKING REQUEST")
print("=" * 65)
```

```
print("Action      :", booking["Action"])
print("Destination :", booking["Destination"])
print("Date        :", booking["Date"])
print("Passengers   :", booking["Passengers"])
```

```
# -----
# 11. ADVANTAGES
# -----
```

```
print("\n" + "=" * 65)
print("ADVANTAGES")
print("=" * 65)
```

```
print("1. Handles incomplete commands.")
print("2. Handles ambiguous phrases.")
print("3. Handles user corrections.")
print("4. Uses context to resolve ambiguity.")
print("5. Feature structures enforce constraints.")
print("6. Earley parsing supports partial input.")
print("7. PCFG helps select the best interpretation.")
```

```
# -----
# FINAL RESULT
# -----
```

```
print("\n" + "=" * 65)
print("FINAL RESULT")
print("=" * 65)
```

```
print("The passenger initially selected Mumbai.")
print("The passenger then corrected it to Bangalore.")
```

```
print("\nFinal destination: Bangalore")
print("Passengers: 2")
print("Date: Tomorrow")
```

```
print("\nThe proposed architecture is suitable")
print("for real-time airline voice assistants.")
```

OUTPUT:

```
IDE Shell 3.10.11
File Edit Shell Debug Options Window Help
>>>
===== RESTART: E1/C06-AT2(QUESTIONS).py =====
INTELLIGENT AIRLINE VOICE ASSISTANT
=====

Speech Input:
Change my flight to Mumbai tomorrow with two passengers.

Tokens:
['change', 'my', 'flight', 'to', 'Mumbai', 'tomorrow', 'with', 'two', 'passengers']

=====
TASK A - SYNTACTIC AMBIGUITY
=====

Correct interpretation:
Change flight
Destination = Mumbai
Date = Tomorrow
Passengers = 2

Incorrect interpretation:
'with two passengers' may be attached
to the destination 'Mumbai'.

=====
FEATURE STRUCTURE
=====

Action : Change Flight
Destination : Mumbai
Date : Tomorrow
Passengers : 2

=====
PCFG AMBIGUITY RESOLUTION
=====

PCFG assigns probabilities to possible parses.
The most probable interpretation is selected.

Selected interpretation:
```

Ln: 557 Col: 0

```
IDE Shell 3.10.11
File Edit Shell Debug Options Window Help
Selected interpretation:
Passengers = 2
Destination = Mumbai

=====
EARLY PARSING
=====

Early parsing can handle:
1. Partial input
2. Ambiguous structures
3. Long-distance dependencies
4. Different sentence structures

=====
TASK B - USER CORRECTION
=====

Original Command:
Change my flight to Mumbai tomorrow.

Correction:
Actually, make that Bangalore.

Updated Booking:
Action : Change Flight
Destination : Bangalore
Date : Tomorrow
Passengers : 2

=====
FEATURE VALIDATION
=====

Passenger count: Valid
Destination: Valid
Travel date: Valid

=====
TASK C & D - IMPROVED ARCHITECTURE
=====
```

Ln: 557 Col: 0

Speech Input

```
IDE Shell 310.11
File Edit Shell Debug Options Window Help
Travel date: Valid

=====
TASK C & D - IMPROVED ARCHITECTURE
=====

Speech Input
  |
  v
Speech Recognition
  |
  v
Tokenization
  |
  v
Earley Parsing
  |
  v
PCFG Ranking
  |
  v
Feature Structures
  |
  v
Context Analysis
  |
  v
Correction Detection
  |
  v
Structured Booking Request
  |
  v
Confirmation
  |
  v
Flight Modification

=====
FINAL BOOKING REQUEST
=====

Structured Booking Request
  |
  v
Confirmation
  |
  v
Flight Modification

=====
FINAL BOOKING REQUEST
=====
Action      : Change Flight
Destination : Bangalore
Date       : Tomorrow
Passengers  : 2

=====
ADVANTAGES
=====
1. Handles incomplete commands.
2. Handles ambiguous phrases.
3. Handles user corrections.
4. Uses context to resolve ambiguity.
5. Feature structures enforce constraints.
6. Earley parsing supports partial input.
7. PCFG helps select the best interpretation.

=====
FINAL RESULT
=====
The passenger initially selected Mumbai.
The passenger then corrected it to Bangalore.

Final destination: Bangalore
Passengers: 2
Date: Tomorrow

The proposed architecture is suitable
for real-time airline voice assistants.
>>>
```

Final output should show

First command:

Change my flight to Mumbai tomorrow.

Correction:

Actually, make that Bangalore.

Updated Booking:

```
{
  'action': 'Change',
  'destination': 'Bangalore',
  'date': 'Tomorrow',
  'passengers': 2
}
```

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1	3	4	4	3	3	3	18	89
2	3	3	4	4	3	3	18	
3	3	3	3	4	4	3	18	
4	4	4	3	3	3	3	17	
5	4	3	4	3	3	3	18	

Faculty In-charge	Course Coordinator	Program Director

Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____